



Trabajo Práctico - Base de Datos II:

Sistema de persistencia políglota para VetSalud

Grupo 9

Integrantes:

Cuneo Gima, Matias - 64492

Canzonieri, Nicolas - 63501

Rossi, Luca Maximiliano - 63730

Fecha de entrega: 12/6/2026

Índice

Decisiones de diseño.....	2
Estructuras de almacenamiento y modelado de datos.....	4
Modelado en MongoDB.....	4
Colección propietarios.....	4
Colección pacientes.....	4
Colección veterinarios.....	5
Colección consultas.....	6
Colección stock.....	7
Vista turnos.....	7
Estructuras utilizadas en Redis.....	8
Sorted Set: vacunas por vencimiento.....	8
Hashes: stock operacional.....	9
Hashes de ingresos por período.....	9
Posibles expansiones y mejoras futuras.....	11
Escalabilidad horizontal.....	11
Sincronización entre MongoDB y Redis.....	11
Ejemplos de llamadas.....	13
Query 1:.....	13
Query 2:.....	15
Query 3:.....	15
Query 4:.....	16
Query 5:.....	17
Query 6:.....	17
Query 7:.....	17
Query 8:.....	18
Query 9:.....	18
Query 10:.....	19
Query 11:.....	19
Query 12:.....	19
Query 13:.....	20
Query 14:.....	21
Query 15:.....	21

Decisiones de diseño

MongoDB fue elegido como base principal por su modelo documental flexible y porque las entidades del dominio (pacientes, propietarios, consultas y veterinarios) se representan naturalmente como documentos. Además, permite evolucionar el esquema de manera sencilla ante futuros cambios en los requerimientos sin necesidad de migraciones complejas.

Dentro del modelo, las vacunaciones fueron embebidas dentro de cada paciente debido a su fuerte relación de pertenencia y a que habitualmente se consultan junto con el historial de la mascota. Al tratarse de una relación de composición y de un conjunto de datos de tamaño acotado, esta decisión mejora la localidad de acceso y evita agregaciones o joins innecesarios.

Los índices fueron diseñados en función de los patrones de acceso identificados en el problema. Se priorizaron búsquedas por paciente, veterinario y fecha mediante índices simples y compuestos, optimizando consultas frecuentes como la obtención del historial clínico de una mascota, la actividad reciente de un veterinario o las consultas realizadas en un determinado período temporal. De esta forma se reducen significativamente los tiempos de búsqueda y ordenamiento sobre grandes volúmenes de registros.

Redis fue incorporado como almacenamiento complementario para optimizar consultas operacionales frecuentes que resultarían más costosas si se resolvieran exclusivamente sobre MongoDB. A diferencia de un simple caché de documentos, Redis se utilizó aprovechando distintas estructuras de datos especializadas según cada necesidad: Sorted Sets para gestionar vacunaciones ordenadas por fecha de vencimiento, Hashes para acceso inmediato al stock farmacéutico y para métricas de ingresos por período y veterinario.

La utilización de Redis permite evitar recorridos completos de colecciones y agregaciones repetitivas sobre MongoDB. Por ejemplo, la consulta de vacunas próximas a vencer puede resolverse eficientemente mediante búsquedas por rango sobre un Sorted Set, mientras que las estadísticas de ingresos pueden obtenerse directamente desde contadores previamente actualizados, eliminando la necesidad de recalcular sumatorias sobre miles de consultas en cada ejecución.

La arquitectura introduce cierta redundancia entre MongoDB y Redis. Sin embargo, esta duplicación se considera aceptable debido a que Redis almacena únicamente proyecciones derivadas y estructuras optimizadas para acelerar consultas frecuentes, mientras que MongoDB conserva el estado completo y actúa como única fuente de verdad del sistema. Ante cualquier inconsistencia o necesidad de reconstrucción, la información almacenada en Redis puede regenerarse a partir de los datos persistidos en MongoDB.

Desde el punto de vista de la consistencia, MongoDB ofrece consistencia fuerte sobre el nodo primario y garantiza la integridad de la información clínica y administrativa del sistema. Redis, por su parte, almacena información derivada cuyo objetivo principal es

mejorar el rendimiento de lectura, por lo que eventuales desincronizaciones temporales tienen un impacto limitado y pueden corregirse reconstruyendo las estructuras auxiliares.

La solución también presenta buenas perspectivas de escalabilidad. MongoDB permite distribuir datos mediante mecanismos de sharding en caso de crecimiento significativo del volumen de información, mientras que Redis puede evolucionar hacia configuraciones distribuidas. Esto permite que la arquitectura pueda crecer manteniendo los mismos modelos de datos y patrones de acceso utilizados inicialmente.

Se descartó Cassandra debido a que el volumen de datos y los requisitos de disponibilidad del problema no justifican la complejidad adicional de modelado y la duplicación de información que requiere una estrategia orientada a consultas. DynamoDB presenta limitaciones similares, requiriendo un diseño basado en claves de partición e índices secundarios específicos para soportar los distintos patrones de acceso identificados. Neo4j tampoco resulta adecuado porque el dominio no presenta relaciones complejas ni recorridos profundos de grafos que permitan aprovechar sus ventajas frente a una base documental.

En consecuencia, la combinación de MongoDB y Redis ofrece el mejor equilibrio entre simplicidad de desarrollo, rendimiento, consistencia y adaptación a los patrones de acceso del problema. MongoDB proporciona una representación natural de las entidades y relaciones del dominio, mientras que Redis permite optimizar consultas críticas mediante estructuras especializadas sin comprometer la integridad de los datos persistentes.

Estructuras de almacenamiento y modelado de datos

La solución implementada utiliza MongoDB como base de datos principal para la persistencia de la información del dominio y Redis como almacenamiento complementario orientado a optimizar consultas operacionales específicas. A continuación se describen las estructuras utilizadas en cada tecnología.

Modelado en MongoDB

MongoDB almacena la información principal del sistema mediante colecciones documentales. Cada colección representa una entidad del dominio veterinario.

Colección propietarios

Almacena la información de los dueños de las mascotas.

Esquema:

```
{
  "_id": "C001",
  "nombre": "Juan",
  "apellido": "Perez",
  "dni": "12345678",
  "email": "juan@email.com",
  "telefono": "1122334455",
  "ciudad": "Buenos Aires",
  "provincia": "Buenos Aires",
  "activo": true
}
```

Índices:

```
{ activo: 1 }
```

Colección pacientes

Almacena las mascotas registradas y su historial de vacunación.

Esquema:

```
{
  "_id": "P001",
  "nombre": "Firulais",
  "especie": "Perro",
  "raza": "Labrador",
  "fecha_nac": "2019-04-12",
  "id_propietario": "C001",
  "activo": true,
  "vacunaciones": [
    {
      "id_vacuna": "V001",
      "id_vet": "VT01",
      "fecha_aplicacion": "2025-01-10",
      "nombre_vacuna": "Antirrábica",
      "proxima_dosis": "2026-01-10"
    }
  ]
}
```

La información de vacunaciones se encuentra embebida dentro de cada paciente debido a la fuerte relación de pertenencia existente entre ambas entidades y a que normalmente se consultan de manera conjunta. Se consideró también la idea de embeber pacientes en propietarios para potencialmente simplificar queries 1, 4 y 12, pero se descartó ante las posibles anomalías que podrían generarse y la gran duplicación de datos.

Índices:

```
{ id_propietario: 1 }
```

Colección veterinarios

Contiene la información profesional de cada veterinario.

Esquema:

```
{
  "_id": "VT01",
```

```
"nombre": "Ana",
"apellido": "Gomez",
"matricula": "MP1234",
"especialidad": "Clínica General",
"sucursal": "Centro",
"activo": true
}
```

Índices:

```
{ sucursal: 1 }
```

Colección consultas

Almacena el historial clínico de las consultas realizadas.

Esquema:

```
{
  "_id": "C0001",
  "id_paciente": "P001",
  "id_vet": "VT01",
  "fecha": "2025-05-20T10:30:00",
  "motivo": "Control",
  "diagnostico": "Sin observaciones",
  "costo": 15000,
  "estado": "Finalizada"
}
```

Índices:

```
{ id_paciente: 1 }
```

```
{ id_vet: 1 }
```

```
{ fecha: -1 }

{ estado: 1 }

{ id_paciente: 1, fecha: -1 }

{ id_vet: 1, fecha: -1 }
```

Los índices compuestos permiten recuperar eficientemente el historial más reciente de una mascota o de un veterinario sin necesidad de ordenar grandes cantidades de registros en memoria.

Colección stock

Almacena los productos farmacéuticos disponibles.

Esquema:

```
{
  "_id": "PR001",
  "nombre": "Vacuna Antirrábica",
  "categoria": "Vacunas",
  "unidades": 150,
  "precio_unit": 2500,
  "vencimiento": "2027-03-10",
  "proveedor": "Laboratorio X"
}
```

Esta colección representa el estado persistente y completo del inventario.

Vista turnos

Unifica en una única fuente temporal los eventos clínicos de un paciente, combinando consultas médicas y vacunaciones ordenables por fecha.

Esquema de un documento resultante:

```
{
  "_id": "C0001",
  "id_paciente": "P001",
```



```
"id_vet": "VT01",  
"fecha": "2025-05-20T10:30:00",  
"tipo": "Consulta"  
}
```

Se implementa como una vista de MongoDB (no una colección física) definida sobre consultas mediante un pipeline que incorpora las vacunaciones embebidas en pacientes a través de \$unionWith. Al no almacenar datos propios, no requiere sincronización manual ni presenta riesgo de anomalías de actualización: cualquier modificación en consultas o en las vacunaciones de pacientes se refleja automáticamente en la vista. Se creó principalmente para agilizar las queries que refieren a visitas generales a la veterinaria.

Estructuras utilizadas en Redis

Redis se utiliza para almacenar información derivada y optimizada para consultas frecuentes. No constituye la fuente principal de verdad del sistema, sino una capa especializada para mejorar el rendimiento.

Sorted Set: vacunas por vencimiento

Clave: vacunas:vencimiento

Tipo: Sorted Set (ZSET)

Score: timestamp de proxima_dosis

Valor:

```
{  
  "id_vacuna": "V001",  
  "id_paciente": "P001",  
  "id_vet": "VT01",  
  "proxima_dosis": "2026-01-10"  
}
```

Esta estructura permite recuperar rápidamente vacunas próximas a vencer utilizando búsquedas por rango sobre fechas.

Hashes: stock operacional

Clave: stock:<id_producto>

Tipo: Hash

Campos:

```
{
  "nombre": "Vacuna Antirrábica",
  "categoria": "Vacunas",
  "unidades": 150,
  "proveedor": "Laboratorio X"
}
```

Los hashes permiten acceder al estado operativo del stock mediante búsquedas directas de complejidad constante.

Hashes de ingresos por período

Clave: stats:ingresos:<año-mes>

Ejemplo: stats:ingresos:2025-05

Tipo: Hash

Campos:

```
{
  "VT01": 450000,
  "VT02": 320000,
  "VT03": 510000
}
```

Cada campo del hash corresponde al identificador de un veterinario y almacena la suma acumulada de ingresos generados durante el período representado por la clave.

Los valores son calculados durante la carga de datos mediante operaciones HINCBY, acumulando el costo de cada consulta sobre el veterinario correspondiente.

Esta estructura permite obtener rápidamente los ingresos de un veterinario para un determinado mes mediante una única lectura, así como recuperar los ingresos de todos los veterinarios de un período sin necesidad de ejecutar agregaciones sobre la colección de consultas almacenada en MongoDB.

Posibles expansiones y mejoras futuras

La arquitectura propuesta fue diseñada para satisfacer los requerimientos actuales del problema manteniendo un equilibrio entre simplicidad, rendimiento y facilidad de implementación. Sin embargo, existen diversas estrategias que permitirían escalar el sistema y mejorar ciertos aspectos operativos en escenarios de mayor volumen de datos o concurrencia.

Escalabilidad horizontal

Una de las principales características de la solución es que las tecnologías elegidas permiten escalar horizontalmente de manera relativamente sencilla y costo-efectiva.

En el caso de MongoDB, el crecimiento del volumen de información puede abordarse mediante mecanismos de replicación y sharding. La replicación permite distribuir las consultas de lectura entre múltiples nodos, mientras que el sharding posibilita particionar los datos entre distintos servidores, incrementando tanto la capacidad de almacenamiento como el rendimiento general del sistema sin necesidad de modificar el modelo de datos.

Por su parte, Redis puede evolucionar hacia una configuración distribuida, donde las claves se distribuyen automáticamente entre distintos nodos. Esto permite aumentar tanto la memoria disponible como la capacidad de procesamiento de consultas. Dado que una parte importante de la optimización implementada se basa en estructuras almacenadas en memoria, esta capacidad resulta especialmente relevante para soportar mayores volúmenes de datos sin degradar los tiempos de respuesta.

Como consecuencia, el elevado consumo de memoria asociado a índices de MongoDB y estructuras especializadas de Redis no representa una limitación significativa a largo plazo, ya que ambas tecnologías ofrecen mecanismos maduros de escalabilidad horizontal que permiten distribuir la carga entre múltiples servidores.

Sincronización entre MongoDB y Redis

Actualmente, ciertas operaciones modifican simultáneamente información almacenada en MongoDB y Redis. Un ejemplo de ello es la actualización del stock farmacéutico, donde una misma operación debe reflejarse tanto en la colección persistente de MongoDB como en los hashes utilizados por Redis para consultas rápidas.

Esta estrategia resulta adecuada para el alcance actual del proyecto debido a su simplicidad de implementación. Sin embargo, presenta algunas limitaciones:

- Incrementa la cantidad de operaciones realizadas por cada actualización.
- Puede generar inconsistencias si una actualización se realiza correctamente en una base de datos pero falla en la otra.

- Parte de la ventaja de rendimiento de Redis se pierde al depender también de la confirmación de MongoDB.

Como mejora futura, podría incorporarse una cola persistente de actualizaciones.

Ejemplos de llamadas

A continuación, se detallan algunos ejemplos de ejecuciones de las queries con sus casos relevantes.

Query 1:

```
node .\queries\q1_pacientes_activos.js
Conectado a MongoDB
[
  {
    _id: 'P001',
    nombre: 'Firulais',
    especie: 'Perro',
    raza: 'Labrador',
    fecha_nac: 2019-04-12T00:00:00.000Z,
    propietario: {
      nombre: 'Ana',
      apellido: 'Rodríguez',
      dni: '30456789',
      email: 'ana@gmail.com',
      telefono: '1145001234',
      ciudad: 'Buenos Aires',
      provincia: 'Buenos Aires'
    }
  },
  {
    _id: 'P002',
    nombre: 'Michi',
    especie: 'Gato',
    raza: 'Siamés',
    fecha_nac: 2021-07-03T00:00:00.000Z,
    propietario: {
      nombre: 'Roberto',
      apellido: 'Díaz',
      dni: '27654321',
      email: 'rob@gmail.com',
      telefono: '1156002345',
      ciudad: 'Rosario',
      provincia: 'Santa Fe'
    }
  },
  {
    _id: 'P004',
    nombre: 'Nube',
    especie: 'Conejo',
    raza: 'Angora',
    fecha_nac: 2022-02-14T00:00:00.000Z,
    propietario: {
      nombre: 'Valentina',
      apellido: 'Molina',
      dni: '41234567',
      email: 'vale@gmail.com',
      telefono: '1167003456',
      ciudad: 'Córdoba',
      provincia: 'Córdoba'
    }
  },
],
```

```

{
  _id: 'P005',
  nombre: 'Rex',
  especie: 'Perro',
  raza: 'Pastor Alemán',
  fecha_nac: 2020-09-08T00:00:00.000Z,
  propietario: {
    nombre: 'Diego',
    apellido: 'Herrera',
    dni: '35789012',
    email: 'diego@gmail.com',
    telefono: '1178004567',
    ciudad: 'Mendoza',
    provincia: 'Mendoza'
  }
},
{
  _id: 'P006',
  nombre: 'Luna',
  especie: 'Gato',
  raza: 'Persa',
  fecha_nac: 2023-01-17T00:00:00.000Z,
  propietario: {
    nombre: 'Paula',
    apellido: 'Sosa',
    dni: '38012345',
    email: 'paula@gmail.com',
    telefono: '1189005678',
    ciudad: 'La Plata',
    provincia: 'Buenos Aires'
  }
},
{
  _id: 'P007',
  nombre: 'Pipo',
  especie: 'Perro',
  raza: 'Poodle',
  fecha_nac: 2017-06-30T00:00:00.000Z,
  propietario: {
    nombre: 'Roberto',
    apellido: 'Díaz',
    dni: '27654321',
    email: 'rob@gmail.com',
    telefono: '1156002345',
    ciudad: 'Rosario',
    provincia: 'Santa Fe'
  }
},
{
  _id: 'P008',
  nombre: 'Bambú',
  especie: 'Ave',
  raza: 'Cacatúa',
  fecha_nac: 2021-03-22T00:00:00.000Z,
  propietario: {
    nombre: 'Tomás',
    apellido: 'Juárez',
    dni: '42345678',
    email: 'tomas@gmail.com',
    telefono: '1190006789',
  }
}

```

```
    ciudad: 'Salta',
    provincia: 'Salta'
  }
}
```

Query 2:

node .\queries\q2_consultas_abiertas.js

Conectado a MongoDB

```
[
  {
    _id: 'CON004',
    id_paciente: 'P004',
    fecha: 2025-11-01T00:00:00.000Z,
    motivo: 'Pérdida de apetito',
    diagnostico: 'Estrés ambiental',
    costo: 3800,
    estado: 'Seguimiento',
    veterinario: {
      nombre: 'Gabriela',
      apellido: 'Fontana',
      matricula: 'VET-0041',
      especialidad: 'Clínica General'
    }
  },
  {
    _id: 'CON005',
    id_paciente: 'P005',
    fecha: 2025-11-10T00:00:00.000Z,
    motivo: 'Vómitos recurrentes',
    diagnostico: 'Gastritis',
    costo: 5500,
    estado: 'Seguimiento',
    veterinario: {
      nombre: 'Lorena',
      apellido: 'Ibáñez',
      matricula: 'VET-0089',
      especialidad: 'Clínica General'
    }
  }
]
```

Query 3:

node .\queries\q3_historial_pacientes.js P001

Conectado a MongoDB

```
{
  "paciente": "Firulais",
  "historial": [
    {
      "tipo": "consulta",
      "fecha": "2025-10-05T00:00:00.000Z",
      "detalle": {
        "_id": "CON001",
        "id_paciente": "P001",
        "id_vet": "V001",
```



```

        "fecha": "2025-10-05T00:00:00.000Z",
        "motivo": "Control anual",
        "diagnostico": "Sano",
        "costo": 4500,
        "estado": "Cerrada"
    },
    {
        "tipo": "vacunacion",
        "fecha": "2025-10-05T00:00:00.000Z",
        "detalle": {
            "id_vacuna": "VAC001",
            "id_vet": "V001",
            "fecha_aplicacion": "2025-10-05T00:00:00.000Z",
            "nombre_vacuna": "Antirrábica",
            "proxima_dosis": "2026-10-05T00:00:00.000Z"
        }
    },
    {
        "tipo": "vacunacion",
        "fecha": "2025-10-05T00:00:00.000Z",
        "detalle": {
            "id_vacuna": "VAC002",
            "id_vet": "V001",
            "fecha_aplicacion": "2025-10-05T00:00:00.000Z",
            "nombre_vacuna": "Sextuple",
            "proxima_dosis": "2026-10-05T00:00:00.000Z"
        }
    },
    {
        "tipo": "consulta",
        "fecha": "2025-12-02T00:00:00.000Z",
        "detalle": {
            "_id": "CON007",
            "id_paciente": "P001",
            "id_vet": "V001",
            "fecha": "2025-12-02T00:00:00.000Z",
            "motivo": "Control post-vacuna",
            "diagnostico": "Sin novedades",
            "costo": 2000,
            "estado": "Cerrada"
        }
    }
]
}

```

Query 4:

```

node .\queries\q4_propietarios_multiples_pacientes.js
Conectado a MongoDB
[
  {
    _id: 'C001',
    cantidad_pacientes: 2,
    pacientes: [ 'Firulais', 'Coco' ],
    propietario: { nombre: 'Ana', apellido: 'Rodríguez', email:
'ana@gmail.com' }
  },
  {

```

```
    _id: 'C002',
    cantidad_pacientes: 2,
    pacientes: [ 'Michi', 'Pipo' ],
    propietario: { nombre: 'Roberto', apellido: 'Díaz', email:
'rob@gmail.com' }
  }
]
```

Query 5:

```
node .\queries\q5_veterinarios_activos_con_consultas_60dias.js
```

Conectado a MongoDB

```
[
  {
    _id: 'V001',
    nombre: 'Gabriela',
    apellido: 'Fontana',
    especialidad: 'Clínica General',
    sucursal: 'Palermo',
    cantidad_consultas: 0
  },
  {
    _id: 'V002',
    nombre: 'Hernán',
    apellido: 'Castillo',
    especialidad: 'Cirugía',
    sucursal: 'Belgrano',
    cantidad_consultas: 0
  },
  {
    _id: 'V003',
    nombre: 'Cecilia',
    apellido: 'Ríos',
    especialidad: 'Dermatología',
    sucursal: 'Palermo',
    cantidad_consultas: 0
  },
  {
    _id: 'V005',
    nombre: 'Lorena',
    apellido: 'Ibáñez',
    especialidad: 'Clínica General',
    sucursal: 'Caballito',
    cantidad_consultas: 0
  }
]
```

Query 6:

```
node .\queries\q6_vacunas_vencidas.js
```

Conectado a Redis

```
[]
```

Query 7:

```
node .\queries\q7_top5_diagnostics.js
```

Conectado a MongoDB

```
[
  { frecuencia: 1, diagnostico: 'Dermatitis atópica' },
  { frecuencia: 1, diagnostico: 'Gastritis' },
  { frecuencia: 1, diagnostico: 'Fractura radio' },
  { frecuencia: 1, diagnostico: 'Sin novedades' },
  { frecuencia: 1, diagnostico: 'Sano' }
]
```

Query 8:

node .\queries\q8_stock_bajo.js

Conectado a MongoDB

```
[
  {
    _id: 'PRD004',
    nombre: 'Frontline Spray',
    categoria: 'Antiparasitario',
    unidades: 40,
    proveedor: 'MediAnimal'
  },
  {
    _id: 'PRD006',
    nombre: 'Ketamina 10%',
    categoria: 'Anestésico',
    unidades: 25,
    proveedor: 'MediAnimal'
  }
]
```

Query 9:

node .\queries\q9_consultas_control.js

Conectado a MongoDB

```
[
  {
    _id: 'CON001',
    id_paciente: 'P001',
    id_vet: 'V001',
    fecha: 2025-10-05T00:00:00.000Z,
    motivo: 'Control anual',
    diagnostico: 'Sano',
    costo: 4500
  },
  {
    _id: 'CON007',
    id_paciente: 'P001',
    id_vet: 'V001',
    fecha: 2025-12-02T00:00:00.000Z,
    motivo: 'Control post-vacuna',
    diagnostico: 'Sin novedades',
    costo: 2000
  }
]
```

Query 10:

```
node .\queries\q10_pacientes_por_sucursal.js Palermo
```

```
Conectado a MongoDB
```

```
[
  {
    _id: 'P004',
    nombre: 'Nube',
    especie: 'Conejo',
    raza: 'Angora',
    fecha_nac: 2022-02-14T00:00:00.000Z,
    id_propietario: 'C003',
    activo: true
  },
  {
    _id: 'P001',
    nombre: 'Firulais',
    especie: 'Perro',
    raza: 'Labrador',
    fecha_nac: 2019-04-12T00:00:00.000Z,
    id_propietario: 'C001',
    activo: true
  },
  {
    _id: 'P002',
    nombre: 'Michi',
    especie: 'Gato',
    raza: 'Siamés',
    fecha_nac: 2021-07-03T00:00:00.000Z,
    id_propietario: 'C002',
    activo: true
  },
  {
    _id: 'P006',
    nombre: 'Luna',
    especie: 'Gato',
    raza: 'Persa',
    fecha_nac: 2023-01-17T00:00:00.000Z,
    id_propietario: 'C005',
    activo: true
  }
]
```

Query 11:

```
node .\queries\q11_vista_ingresos.js
```

```
Conectado a Redis
```

```
Conectado a MongoDB
```

```
[]
```

Query 12:

```
node .\queries\q12_propietarios_sin_consultas.js
```

```
Conectado a MongoDB
```

```
[
  {
    _id: 'C006',
```

```
nombre: 'Tomás',
apellido: 'Juárez',
dni: '42345678',
email: 'tomas@gmail.com',
telefono: '1190006789',
ciudad: 'Salta',
provincia: 'Salta',
activo: true
}
]
```

Query 13:

```
node .\queries\q13_delete_propietario.js C001
```

Conectado a MongoDB

```
{ _id: 'C001' }
```

```
node .\queries\q13_insert_propietario.js C038 Matias Cuneo 99 a@a.com 54 ba ba
```

Conectado a MongoDB

```
{
  _id: 'C038',
  nombre: 'Matias',
  apellido: 'Cuneo',
  dni: '99',
  email: 'a@a.com',
  telefono: '54',
  ciudad: 'ba',
  provincia: 'ba'
}
```

```
node .\queries\q13_update_propietario.js C038 Matias Cuneo 99 a@a.com 54 bahia ba
```

Conectado a MongoDB

```
{
  _id: 'C038',
  nombre: 'Matias',
  apellido: 'Cuneo',
  dni: '99',
  email: 'a@a.com',
  telefono: '54',
  ciudad: 'bahia',
  provincia: 'ba'
}
```

Query 14:

```
node .\queries\q14_nueva_consulta.js CON020 P001 V001 2023-10-10
Consulta-de-rutina Diagnóstico-de-rutina 100 Seguimiento
Conectado a MongoDB
{
  _id: 'CON020',
  id_paciente: 'P001',
  id_vet: 'V001',
  fecha: '2023-10-10',
  motivo: 'Consulta-de-rutina',
  diagnostico: 'Diagnóstico-de-rutina',
  costo: '100',
  estado: 'Seguimiento'
}
```

Query 15:

```
node .\queries\q15_decremento_stock.js PRD001 10
Conectado a Redis
Conectado a MongoDB
Stock actualizado. Producto: PRD001 | Unidades descontadas: 10 | Stock
restante: 110
```

```
node .\queries\q15_decremento_stock.js PRD001 10 (segunda llamada)
Conectado a Redis
Conectado a MongoDB
Stock actualizado. Producto: PRD001 | Unidades descontadas: 10 | Stock
restante: 100
```